Ticket Booking Manager - Project Documentation

Alireza Nobakht-Mohammad Taghi Ghaffari-Mahan Nojavan

Table of Contents

Project Overview

The Ticket Booking Manager is a backend server application designed to manage ticket bookings for transportation such as planes, trains, and buses. It supports:

- **User signup and login via OTP (One-Time Password).**
- **Secure JWT-based authentication.**
- **Profile management.**
- **Ticket search with flexible filters.**
- **Viewing detailed ticket information.**
- **Reserving tickets with time-limited validity and automatic expiration.**
- **Viewing active reservations and reservation history.**
- **Redis caching for improved performance.**

System Architecture

- **Framework: Flask (Python) for RESTful API development.**
- **Database: MySQL for persistent storage of users, tickets, reservations, etc.**
- **Caching: Redis to cache frequently accessed data like ticket search results.**
- **Authentication: JWT tokens for stateless and secure authentication.**
- **Modular Design: Uses Flask Blueprints to organize APIs by domain (auth, tickets, reservations, cities).**
- **Service Layer: Business logic separated from API routes for maintainability.**
- **Database Abstraction: Raw SQL queries encapsulated in a dedicated database layer.**
- **Logging: Extensive use of Python's logging module for debugging and monitoring.**

Setup and Installation

Prerequisites

- **Python 3.8 or higher**
- **MySQL server installed and running**
- **Redis server installed and running**
- **Required Python packages (listed in requirements.txt)**

Steps

1. **Clone the repository locally.**
2. **Configure database and Redis connection settings in config.py.**
3. **Install dependencies:**

bash

pip install -r requirements.txt

4. **Run the Flask application:**

bash

python app.py

5. **Access the API at http://localhost:5000.**

Configuration

- **Database: Set MySQL host, user, password, and database name in config.py.**
- **Redis: Set Redis host, port, and password if applicable in config.py.**
- **JWT: Configure secret keys and token expiration settings as needed.**

API Reference

1. Authentication APIs

- **Request OTP**
    - **POST /api/auth/login/request-otp**
    - **Body: { "email": "user1@example.com" }**
    - **Sends OTP to user.**
- **Verify OTP**
    - **POST /api/auth/login/verify-otp**
    - **Body: { "email": "user1@example.com", "otp": "278895" }**
    - **Returns JWT token on success.**
- **Sign Up**
    - **POST /api/auth/signup**
    - **Body:**

json

{

"first_name": "Ali",

"last_name": "Rezaei",

"email": "ali.rezaei@example.com",

"phone_number": "09123456789",

"city": "Tehran",

"password": "SuperSecret123"

}

- Update Profile
    - POST /api/auth/profile/update
    - Headers: Authorization: Bearer <token>
    - Body:

json

{

"first_name": "Asoi",

"last_name": "Rei",

"email": "kkkkali.reza@example.com",

"phone_number": "0913456789",

"city": "Tehran"

}

---

2. City API

- Get Cities
    - GET /api/cities
    - Returns list of cities.

---

3. Ticket APIs

- Search Tickets
    - GET /api/tickets/search
    - Query parameters example: origin_id=28&destination_id=45&travel_date=2025-05-10

- **Optional**
  filters: vehicle_type, min_price, max_price, company_name, departure_start, departure_end, travel_class.

- **Ticket Details**
  - **GET /api/tickets/details/<ticket_id>**

---

4. Reservation APIs

- **Reserve Ticket**
  - **POST /api/reservations/reserve**
  - **Headers: Authorization: Bearer <token>**
  - **Body:**

json

{

"ticket_id": 123,

"quantity": 2,

"reservation_time": 10

}

- **Get Active Reservations**
  - **GET /api/reservations/active**
  - **Headers: Authorization: Bearer <token>**
- **Get Reservation History**
  - **GET /api/reservations/history**
  - **Headers: Authorization: Bearer <token>**

Testing the APIs

Using Postman

- **Create requests for each endpoint with proper headers and body.**

- **Use environment variables to store JWT tokens for authenticated requests.**
- **Send requests and verify responses.**

Using curl

Example to request OTP:

bash

```
curl -X POST http://localhost:5000/api/auth/login/request-otp \
 -H "Content-Type: application/json" \
 -d '{"email": "user@example.com"}'
```

Example to reserve ticket:

bash

```
curl -X POST http://localhost:5000/api/reservations/reserve \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer <your_jwt_token>" \
 -d '{"ticket_id": 123, "quantity": 2, "reservation_time": 10}'
```

Code Structure and Explanation

Project Layout

text

```
/api/          # Flask blueprints for API endpoints
/services/     # Business logic services
/db/           # Database access functions
/utils/        # Utility functions (e.g., JWT handling)
/config.py     # Configuration settings
/app.py        # Flask app factory and blueprint registration
```

API Layer (Blueprints)

- **Routes handle HTTP requests, parse inputs, verify JWT tokens, and call service functions.**
- **Example from reservation.py:**

python

```
@reservation_bp.route('/reserve', methods=['POST'])
def reserve_ticket():
  token = request.headers.get('Authorization')
  if token and token.startswith('Bearer '):
    token = token[7:]
  user_data = verify_jwt_token(token)
  data = request.get_json()
  ticket_id = data.get('ticket_id')
  validity_minutes = data.get('validity_minutes', 10)
  reservation_id = reserve_ticket_service(user_data['user_id'], ticket_id, validity_minutes)
  return jsonify({'status': 'success', 'reservation_id': reservation_id})
```

Service Layer

- **Contains business logic and orchestrates database and cache calls.**
- **Example from reservation_service.py:**

python

```
def reserve_ticket_service(passenger_id, ticket_id, validity_minutes=10):
  reservation_id = create_reservation(passenger_id, ticket_id, validity_minutes)
  return reservation_id
```

Database Layer

- **Uses parameterized SQL queries to prevent injection.**

- **Manages DB connections and cursors safely.**
- **Example from db.py:**

python

```
def create_reservation(passenger_id, ticket_id, validity_minutes=10):
  conn = get_db_connection()
  try:
    with conn.cursor() as cursor:
      expiry_time = datetime.utcnow() + timedelta(minutes=validity_minutes)
      sql = """
      INSERT INTO reservation (passenger_id, reservation_date, status, expiry_time, created_at, updated_at)
      VALUES (%s, %s, %s, %s, %s, %s)
      """
      now = datetime.utcnow()
      cursor.execute(sql, (passenger_id, now, 'active', expiry_time, now, now))
      return cursor.lastrowid
  finally:
    conn.close()
```

Authentication and JWT

- **JWT tokens encode user info and expiry.**
- **Tokens are verified on protected routes.**
- **Token extracted from Authorization header:**

python

```
token = request.headers.get('Authorization')
if token and token.startswith('Bearer '):
```

```
    token = token[7:]
user_data = verify_jwt_token(token)
```

Caching Layer

- **Redis caches search results to improve performance.**
- **Cache keys generated from query parameters.**
- **Cached data serialized to JSON with datetime converted to strings.**

Logging and Error Handling

- **Uses Python logging module at DEBUG level for detailed tracing.**
- **Logs include request data, SQL queries, JWT payloads, and errors.**
- **API responses consistently include status and error messages.**

Additional Notes

- **Ensure Redis and MySQL servers are running and accessible.**
- **JWT tokens expire; re-authenticate as needed.**
- **Reservation expiration is handled automatically after the specified reservation time.**
- **Use consistent UTC timestamps to avoid timezone issues.**
- **Follow modular design for maintainability and scalability.**